

DR Esam Podcast - Complete Documentation

Table of Contents

- 1. [Overview](#)
- 2. [Features](#)
- 3. [System Architecture](#)
- 4. [Getting Started](#)
- 5. [Public Website](#)
- 6. [Admin Dashboard](#)
- 7. [Database Schema](#)
- 8. [API Endpoints](#)
- 9. [Workflows](#)
- 10. [Troubleshooting](#)

Overview

DR Esam Podcast is a comprehensive podcast management and hosting platform built with modern web technologies. It provides tools for podcast content management, audience engagement, subscriber tracking, service request management, and analytics.

Key Features:

- Multi-language support (English & Arabic)
- Dynamic episode management with Supabase integration
- Service request tracking with two-stage approval workflow
- Cross-device content synchronization
- Analytics and revenue reporting
- Newsletter subscription system
- Email automation with Mailgun

Tech Stack:

- Frontend: HTML5, CSS3, Vanilla JavaScript
- Backend: Netlify Serverless Functions (Node.js)
- Database: Supabase (PostgreSQL)
- Storage: Supabase Storage (S3-compatible)
- Email Service: Mailgun
- Deployment: Netlify with GitHub integration

Features

1. Episode Management

- Create, edit, and delete podcast episodes
- Upload episode images to Supabase Storage or use URLs
- Store episode metadata: title, guest name, description
- Track episode view counts
- Link to YouTube and TikTok content

2. Multi-Language Support

- Bilingual interface (English/العربية)
- Content toggles between EN/AR with single click
- All user-facing text supports both languages
- RTL (right-to-left) support for Arabic

3. Service Requests & Revenue Tracking

- Contact form accepts service inquiries
- Two-stage approval workflow:
 - **Discussion** (incoming, not counted as revenue)
 - **Agreed** (approved, counted as revenue)
 - **Done** (completed, counted as revenue)
- Revenue aggregation by service type
- Cost and profit calculations
- Historical reporting with date filtering

4. Cross-Device Content Sync

- Admin updates sync automatically to all devices
- Settings, hero section, about, platforms, and services
- Supabase-backed synchronization
- Hybrid approach: instant localStorage + background Supabase sync
- Works with language toggle

5. Subscriber Management

- Newsletter signup form
- Country-based subscriber tracking
- Subscriber list with export (CSV/JSON)
- Bulk email sending to subscribers
- Confirmation emails via Mailgun

6. Analytics Dashboard

- Total website pageviews
- Episode play/click tracking
- Subscriber growth by country
- Interactive charts (Chart.js)
- Date range filtering

7. Admin Panel

- Secure password-protected access
- Tab-based navigation
- Settings management (social links, credentials)
- Hero section editor
- About section editor
- Platform link management
- Service pricing tiers and add-ons
- Episode CRUD operations
- Subscriber list view
- Service request management
- Analytics dashboard
- Revenue reporting

System Architecture

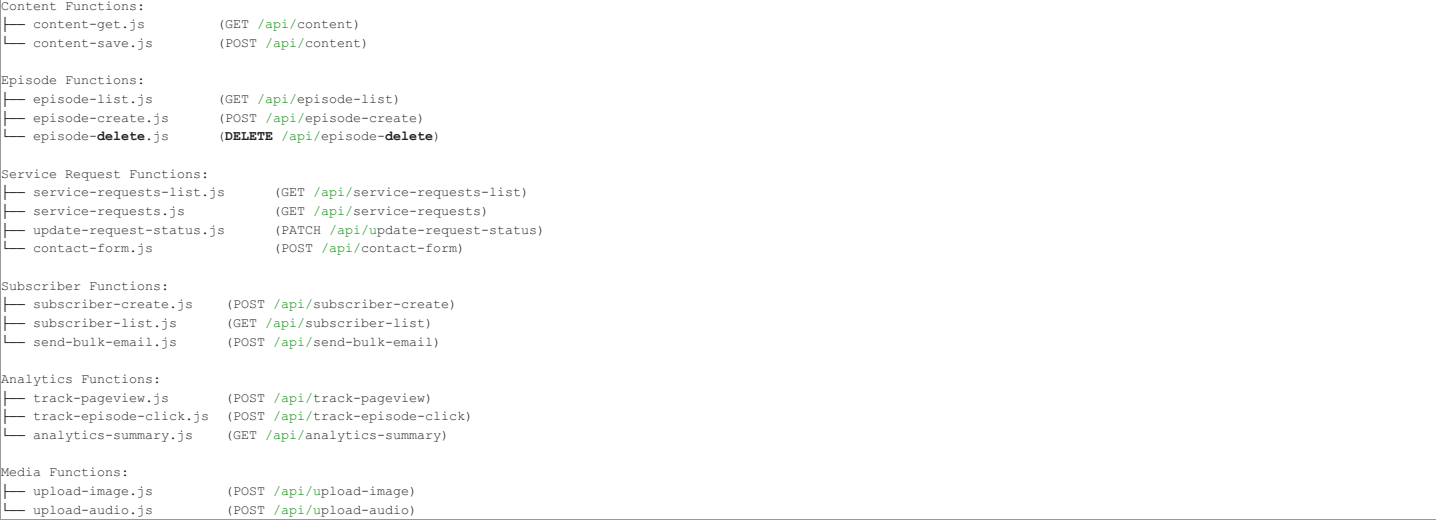
Frontend Layer



Admin Dashboard



Backend Layer (Netlify Functions)



Database Layer (Supabase PostgreSQL)



Getting Started

Prerequisites

- Node.js 14+
- Git
- Netlify account
- Supabase account
- Mailgun account
- GitHub account

Installation

1. Clone Repository

```
git clone https://github.com/your-org/dr-esam-podcast.git
cd dr-esam-podcast
```

2. Set Environment Variables (in Netlify)

```
SUPABASE_URL=your_supabase_url
SUPABASE_KEY=your_supabase_service_role_key
MAILGUN_API_KEY=your_mailgun_api_key
MAILGUN_DOMAIN=your_mailgun_domain
ADMIN_PASSWORD=your_admin_password (default: DrEsam2025)
ANTHROPIC_API_KEY=your_claude_api_key (optional)
```

3. Initialize Database

Run in Supabase SQL Editor:

```
-- Create site_content table
CREATE TABLE site_content (
  key TEXT PRIMARY KEY,
  value JSONB NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Create service_requests table
CREATE TABLE service_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name TEXT,
  last_name TEXT,
  email TEXT,
  subject TEXT,
  service_key TEXT,
  service_name TEXT,
  price_aed DECIMAL,
  cost_aed DECIMAL,
  markup_aed DECIMAL,
  status TEXT DEFAULT 'discussion',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Add status column with update policy
ALTER TABLE service_requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY "allow_select" ON service_requests FOR SELECT TO public USING (true);
CREATE POLICY "allow_insert" ON service_requests FOR INSERT TO public WITH CHECK (true);
CREATE POLICY "allow_update" ON service_requests FOR UPDATE TO public USING (true);
```

4. Deploy

```
npm run deploy
```

Public Website

Homepage

- Hero section with podcast overview
- Statistics (episode count, listeners, rating)
- Bilingual support with language toggle

Episodes Section

- Grid of podcast episodes
- Episode cards with:
 - Image/thumbnail
 - Title and description
 - Guest information
 - Play/listen buttons (YouTube, TikTok links)
 - Episode click tracking

Services Section

- Production tiers (Audio, Standard Video, Cinematic, AI Enhanced, Premium)
- Add-on services (Short Clips, Subtitles, Thumbnails, etc.)
- Pricing in AED currency
- "Get a Quote" CTA (WhatsApp integration)

About Section

- Host biography
- Professional credentials
- Social media links

Contact/Service Request Form

- Name, email, subject (service selection)
- Message field
- Form validation
- Automatic email notification to admin
- Service request created in database with "discussion" status

Subscriber Newsletter

- Email signup form
- Double opt-in via Mailgun confirmation
- Country detection (optional)
- Newsletter management

Footer

- Social media links
- Platform links (Spotify, Apple Podcasts, YouTube, etc.)

Admin Dashboard

Login

- Password-protected access
- Default password: DrEsam2025
- Password change functionality
- Persistent session via localStorage

Dashboard Tabs

1. Settings

- Site name (EN/AR)
- Meta description
- Social media handles (WhatsApp, Instagram, YouTube, TikTok, Twitter)
- API keys (Claude, Google, Replicate)

2. Hero Section

- Title (EN/AR)
- Tagline (EN/AR)
- Accent label and title
- Statistics (episodes, listeners, rating)

3. About

- Host name (EN/AR)
- Role/title (EN/AR)
- Biography (EN/AR)
- Credentials (EN/AR)

4. Platforms

- Links to: Spotify, YouTube, TikTok, Instagram, Apple Podcasts, Google Podcasts

5. Services

- Pricing tiers (5 main services + 6 add-ons)
- Price in USD and AED
- CTA text (EN/AR)

6. Episodes

- List of all episodes
- Create/edit/delete episodes
- Image upload or URL input
- YouTube/TikTok links
- Guest name, description
- Bulk operations

7. Dialogue AI

- Generate podcast scripts using Claude API
- Episode details input
- Script generation and editing

8. Image Generation

- Generate episode thumbnails
- AI image generation via Replicate
- Custom thumbnail creation

9. Podcast Video

- Video generation via HeyGen API
- Avatar-based video creation
- Upload and manage videos

10. Subscribers

- View all newsletter subscribers
- Export as CSV or JSON
- Send bulk emails
- Search/filter functionality

11. Service Requests

- View all incoming service inquiries
- Status: Discussion → Agreed → Done
- Change status with one click
- View inquiry details (name, email, service, message)
- Status color-coded (gray, gold, green)

12. Analytics

- Total website pageviews
- Episode play counts
- Subscriber growth by country
- Interactive charts
- Date range filtering

13. Revenue Report

- Revenue aggregation by service
- Cost and profit breakdown
- Total revenue (only "agreed" and "done" requests)
- Service-wise revenue distribution
- Date range filtering
- Export to CSV/PDF

Database Schema

episodes

id	UUID	PRIMARY KEY
title	TEXT	
quest_name	TEXT	
description	TEXT	
image_url	TEXT	
audio_url	TEXT	
youtube	TEXT	
tiktok	TEXT	
view_count	INT	DEFAULT 0
created_at	TIMESTAMPTZ	

subscribers

id	UUID	PRIMARY KEY
email	TEXT	UNIQUE
name	TEXT	
country	TEXT	
subscribed_at	TIMESTAMPTZ	

service_requests

id	UUID	PRIMARY KEY
first_name	TEXT	
last_name	TEXT	
email	TEXT	
subject	TEXT	
service_key	TEXT	
service_name	TEXT	
price_aed	DECIMAL	
cost_aed	DECIMAL	
markup_aed	DECIMAL	
status	TEXT	(discussion agreed done)
created_at	TIMESTAMPTZ	

pageviews

id	UUID	PRIMARY	KEY
page	TEXT		
visited_at	TIMESTAMP		

site_content

key	TEXT	PRIMARY	KEY
value	JSONB		
updated_at	TIMESTAMP		

API Endpoints

Content Management

- GET /.netlify/functions/content-get — Fetch site content
- POST /.netlify/functions/content-save — Save site content (password protected)

Episodes

- GET /.netlify/functions/episode-list — Get all episodes
- POST /.netlify/functions/episode-create — Create episode
- DELETE /.netlify/functions/episode-delete — Delete episode

Service Requests

- GET /.netlify/functions/service-requests-list — Get all requests (admin)
- GET /.netlify/functions/service-requests — Get revenue summary (filtered by status)
- POST /.netlify/functions/contact-form — Create new service request
- PATCH /.netlify/functions/update-request-status — Update request status

Subscribers

- GET /.netlify/functions/subscriber-list — List all subscribers
- POST /.netlify/functions/subscriber-create — Create new subscriber
- POST /.netlify/functions/send-bulk-email — Send email to subscribers

Analytics

- POST /.netlify/functions/track-pageview — Track page visit
- POST /.netlify/functions/track-episode-click — Track episode play
- GET /.netlify/functions/analytics-summary — Get analytics data

Media

- POST /.netlify/functions/upload-image — Upload image to storage
- POST /.netlify/functions/upload-audio — Upload audio file

Workflows

Service Request Workflow

1. User submits contact form
↓
2. Contact-form function processes:
- Validates input
- Sends email to admin via Mailgun
- Inserts request with status='discussion'
↓
3. Request appears in Admin → Service Requests tab
(Status: DISCUSSION - gray badge, not counted as revenue)
↓
4. Admin reviews inquiry
↓
5. Admin clicks "- Agreed" or "- Done"
↓
6. Request status updated in database
↓
7. Request now visible in Admin → Revenue Report
(Counted as revenue with price_aed, cost_aed, profit)

Content Sync Workflow

1. Admin updates settings/hero/about/platforms/services
↓
2. Admin clicks "Save All"
↓
3. Content saved to:
a) localStorage (instant, for offline access)
b) Supabase via content-save.js (background sync)
↓
4. Public site fetches from Supabase (or localStorage fallback)
↓
5. All devices show updated content instantly
(No refresh needed on other devices after sync completes)

Episode Management Workflow

1. Admin clicks "Add Episode" or edits existing
↓
2. Fill in episode details:
- Title, guest name, description
- Upload image OR paste Supabase Storage URL
- YouTube and TikTok links
↓
3. Click "Save Episode"
↓
4. Episode created/updated in Supabase
↓
5. Episode appears on public site Episodes section
↓
6. Public site loads episodes via episode-list.js
↓
7. Episode view counts tracked via track-episode-click.js

Troubleshooting

Admin Panel Issues

"Content not syncing to other devices"

- Ensure Supabase credentials are set in Netlify environment
- Check browser console for fetch errors
- Try hard refresh (Ctrl+Shift+R) on other device
- Verify site_content table exists in Supabase

"Arabic content not showing after language toggle"

- Hard refresh the page
- Clear browser cache
- Ensure Arabic fields are filled in admin settings
- Check that document.documentElement.lang is set correctly

"Service requests not appearing in Revenue Report"

- Verify request status is "agreed" or "done" (not "discussion")
- Check that price_aed field is populated
- Ensure allow_update RLS policy exists on service_requests table

"Episodes not uploading"

- Check image file size (recommended < 2MB)
- Verify Supabase Storage is configured
- Check SUPABASE_URL and SUPABASE_KEY environment variables
- Alternatively, upload to Supabase manually and paste URL

Database Issues

"RLS policy error when updating requests"

```
-- Add missing update policy
CREATE POLICY "allow_update" ON service_requests
FOR UPDATE TO public USING (true);
```

"Content table doesn't exist"

```
-- Recreate table
CREATE TABLE IF NOT EXISTS site_content (
  key TEXT PRIMARY KEY,
  value JSONB NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

Deployment Issues

"Functions returning 404"

- Verify netlify.toml has correct function paths
- Check that functions are in netlify/functions/ directory
- Ensure function names match API endpoints
- Run npx netlify-cli deploy --prod -d . with skip-functions-cache

"Environment variables not found"

- Set variables in Netlify Site Settings → Build & Deploy → Environment
- Restart/redeploy after adding variables
- Verify variable names exactly match function expectations

"CORS errors"

- Check that headers in functions include CORS headers:

```
const headers = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Methods': 'GET, POST, PATCH, OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type'
};
```

Performance Tips

1. Image Optimization

- Use Supabase Storage for images (automatic CDN)
- Compress images before upload (< 500KB recommended)
- Use WebP format when possible

2. Database Queries

- Episode list queries are indexed by created_at
- Service request queries filtered by status for performance
- Use date ranges in revenue reports to limit data

3. Frontend Caching

- Content cached in localStorage for offline access
- Images cached via Netlify CDN
- Supabase REST API results cached in memory

4. Email Optimization

- Bulk emails sent asynchronously (non-blocking)
- Mailgun handles rate limiting
- Template-based emails for consistency

Security Considerations

1. Admin Access

- Password-protected dashboard
- Default password should be changed
- Credentials stored in Supabase environment variables

2. Data Privacy

- Subscriber emails encrypted in Supabase
- RLS policies enforce data access rules
- Service requests visible only to admin (via password)

3. API Security

- Service role key used for sensitive operations
- Password validation on content-save.js
- CORS headers configured for origin validation

4. File Upload

- File uploads to Supabase Storage with signed URLs
- Image validation on upload
- Size limits enforced by Netlify functions

Support & Maintenance

Regular Tasks

- Monitor Supabase storage usage
- Review analytics dashboard monthly
- Export revenue reports for accounting
- Update service pricing as needed
- Backup content regularly (via Supabase backups)

Update Process

1. Make changes in admin dashboard
2. Changes sync to Supabase automatically
3. Public site fetches updated content
4. No deployment needed for content changes

Monitoring

- Check Netlify function logs for errors
- Monitor Supabase query performance
- Review Mailgun email delivery logs
- Track website analytics for trends

Last Updated: April 2026

Version: 1.0.0

Maintained By: Dr. Esam Podcast Team